

Raport Końcowy z Projektu

Przedmiot: Integracja Systemów Informatycznych

Nazwa Projektu: EsportHub

Skład Zespołu: Miłosz Samotyjak

1. Opis Projektu

EsportHub to backendowe REST API do zarządzania turniejem e-sportowym. System obsługuje pełny cykl życia rozgrywek - od rejestracji drużyn i zawodników, przez fazę grupową, aż po fazę pucharową i finał. Aplikacja integruje się z platformą Twitch, umożliwiając tworzenie klipów oraz zarządzanie harmonogramem transmisji na żywo. Integracja jest dodatkową funkcjonalnością aplikacji a sama aplikacja nie jest zależna od tej integracji - główne funkcjonalności aplikacji dalej działają w wypadku przerwania działania Twitch API.

Funkcjonalności:

- tworzenie i zarządzanie drużynami oraz składami zawodników,
- tworzenie turniejów i obsługa ich pełnego cyklu: faza grupowa -> faza pucharowa,
- ustawianie wyników meczów z automatycznym wyznaczaniem zwycięzcy,
- integracja z Twitch API: autoryzacja OAuth 2.0, tworzenie klipów (krótkich filmów obejmujących część transmisji która upłynęła na kilkadziesiąt sekund przed zgłoszeniem prośby utworzenia klipu) oraz harmonogramów transmisji.

2. Architektura Systemu

- **Backend:** ASP.NET 10 Minimal API,
- **Baza danych:** PostgreSQL 18,
- **ORM:** Entity Framework 10,
- **Result Pattern,**
- **konteneryzacja:** Docker + Docker Compose,
- **wdrożenie:** Render.com

3. Realizacja CI/CD

GitHub Actions (CI)

Skonfigurowane zostały 3 workflowy. Wspólna logika wyodrębniona jest do workflowu wielokrotnego użytku `ci-base.yaml`, który jest wywoływany przez pozostałe dwa.

Workflow bazowy `ci-base.yaml` - uruchamiany przy każdym PR i pushu do `main`:

```
name: CI base

on:
```

```
workflow_call:

jobs:
  ci_base:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v5

      - name: Set up .NET
        uses: actions/setup-dotnet@v4
        with:
          dotnet-version: 10.0.x
          cache: true
          cache-dependency-path: EsportHub.Backend/EsportHub.Backend.slnx

      - name: Install dependencies
        run: dotnet restore EsportHub.Backend/EsportHub.Backend.slnx

      - name: Lint
        run: dotnet format EsportHub.Backend/EsportHub.Backend.slnx --
verify-no-changes --no-restore

      - name: Build
        run: dotnet build EsportHub.Backend/EsportHub.Backend.slnx --no-
restore

      - name: Run tests
        run: dotnet test EsportHub.Backend/EsportHub.Backend.slnx --no-
build

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v4

      - name: Build without push
        uses: docker/build-push-action@v7
        with:
          context: EsportHub.Backend
          file: EsportHub.Backend/Dockerfile
          push: false # Only check if build succeeds and ignore build
result

      cache-from: type=gha
      cache-to: type=gha,mode=max
```

Kroki w kolejności:

1. `dotnet restore` - instalacja/przywrócenie bibliotek,
2. `dotnet format --verify-no-changes` - linter, build kończy się błędem przy niezgodności,
3. `dotnet build` - kompilacja,
4. `dotnet test` - uruchomienie testów,
5. Docker build - weryfikacja czy produkcyjny obraz Docker buduje się poprawnie, bez publikowania i zapisywania wyniku.

Zrealizowano testy jednostkowe i integracyjne z pokryciem na poziomie 60%.

Deployment (CD)

Wdrożenie uruchamiane jest automatycznie po każdym pushu do gałęzi `main`, po pomyślnym zakończeniu CI:

```
name: EsportHub CI/CD

on:
  push:
    branches: [main]

jobs:
  ci_base:
    uses: ../github/workflows/ci-base.yaml

  deploy:
    needs: ci_base
    runs-on: ubuntu-latest

    steps:
      - name: Deploy to Render
        env:
          DEPLOY_HOOK: ${ secrets.RENDER_DEPLOY_HOOK_URL }
        run: |
          curl -X POST "$DEPLOY_HOOK"

      - name: Deployment health check
        timeout-minutes: 10
        run: |
          URL="https://esporthub-lpx3.onrender.com/healthz"
          TIMEOUT=600
          INTERVAL=30
          ELAPSED=0

          while [ $ELAPSED -lt $TIMEOUT ]; do
            sleep 30

            STATUS=$(curl -s -m 5 -o /dev/null -w "%{http_code}" $URL ||
true)

            if [ $STATUS -eq 200 ]; then
              exit 0
            fi

            ELAPSED=$((ELAPSED + INTERVAL))
          done

          exit 1
```

1. deploy hook wysyła żądanie POST do Render.com, które inicjuje wdrożenie nowej wersji aplikacji,

2. health check sprawdza co 30 sekund endpoint `/healthz` aplikacji, czekając maksymalnie 10 minut na pomyślną odpowiedź. Jeśli aplikacja nie uruchomi się w tym czasie, krok kończy się błędem,
3. `RENDER_DEPLOY_HOOK_URL` przechowywany jest jako GitHub Actions Secret - nie jest widoczny w kodzie repozytorium.

4. Zarządzanie Projektem (Git/GitHub)

- **Zastosowany Workflow:** GitHub Actions workflow, każda funkcjonalność rozwijana na osobnej gałęzi feature, mergowana do `main` przez Pull Request. Brak możliwości mergowania PR dla CI zakończonych niepowodzeniem. Zarządzanie zadaniami przez GitHub Issues i GitHub Projects.
- **Statystyki PR:** 12 PR, wystawianych i przeglądanych przez samego siebie.
- **Link do repozytorium:** github.com/Me-Wosh/EsportHub

5. Dokumentacja Techniczna (Markdown)

Najważniejsze punkty `README.md`:

- **opis projektu** - czym jest EsportHub i jakie funkcjonalności oferuje,
- **stos technologiczny** - zestawienie tabelaryczne wszystkich użytych technologii,
- **instrukcja uruchomienia w Dockerze** - kroki konfiguracji `.env`, uruchomienia `docker compose up --build` oraz automatycznego nakładania migracji przy starcie,
- **Diagram ERD**,
- **Domain-Driven Design i bogata domena** - opis enkapsulacji reguł biznesowych,
- **Result Pattern** - opis zastosowania wzorca Result jako alternatywy do rzucania wyjątków, zalety i wady takiego podejścia, przykład kodu,
- **Table Per Hierarchy (TPH)** - opis strategii mapowania dziedziczenia Match, konfiguracja dyskryminatora, uzasadnienie wyboru TPH,
- **DeleteBehavior.Restrict** - opis blokowania kaskadowego usuwania encji i uzasadnienie decyzji projektowej,
- **Mediator Pattern** - opis wzorca Mediator, przykład kodu.

6. Podsumowanie i Wnioski

Co udało się zrealizować:

- kompletne REST API z obsługą pełnego cyklu turnieju e-sportowego,
- bogata warstwa domenowa z regułami biznesowymi i Result Pattern,
- pełny pipeline CI/CD: linter, build, testy, Docker build, automatyczne wdrożenie na Render.com,
- integracja z zewnętrznym API (Twitch OAuth 2.0, klipy, harmonogram transmisji),
- konteneryzacja zarówno dla środowiska produkcyjnego (wieloetapowy Dockerfile), jak i deweloperskiego (hot reload przez `dotnet watch`).

Główne wyzwania:

- zaprojektowanie bogatej domeny z regułami biznesowymi, decyzje jak przebiegają przepływy w aplikacji i która odpowiedzialność należy do którego agregatu,
- integracja z platformą Twitch - przestudiowanie dokumentacji i implementacja uwierzytelniania za pomocą Twitch OAuth,

- konfiguracja środowiska deweloperskiego działającego w Dockerze - podpięcie debugera pod działający kontener i hotreloading zmian w kontenerze.

Plany na dalszy rozwój:

- implementacja systemu autentykacji i autoryzacji,
- statystyki (drużyny, gracza).